

Database Design & Normalization

Name: Prajwal

USN: 4SF24CS139

Class: 4A

Scenario 1: Online Learning Platform

1. Introduction

This system represents an online learning platform where instructors create courses and students enroll in them. Each course is divided into modules and lessons. The system also supports tracking student progress, course reviews, and tagging of courses.

The main objective is to design a structured database that avoids redundancy and follows normalization principles up to 3NF.

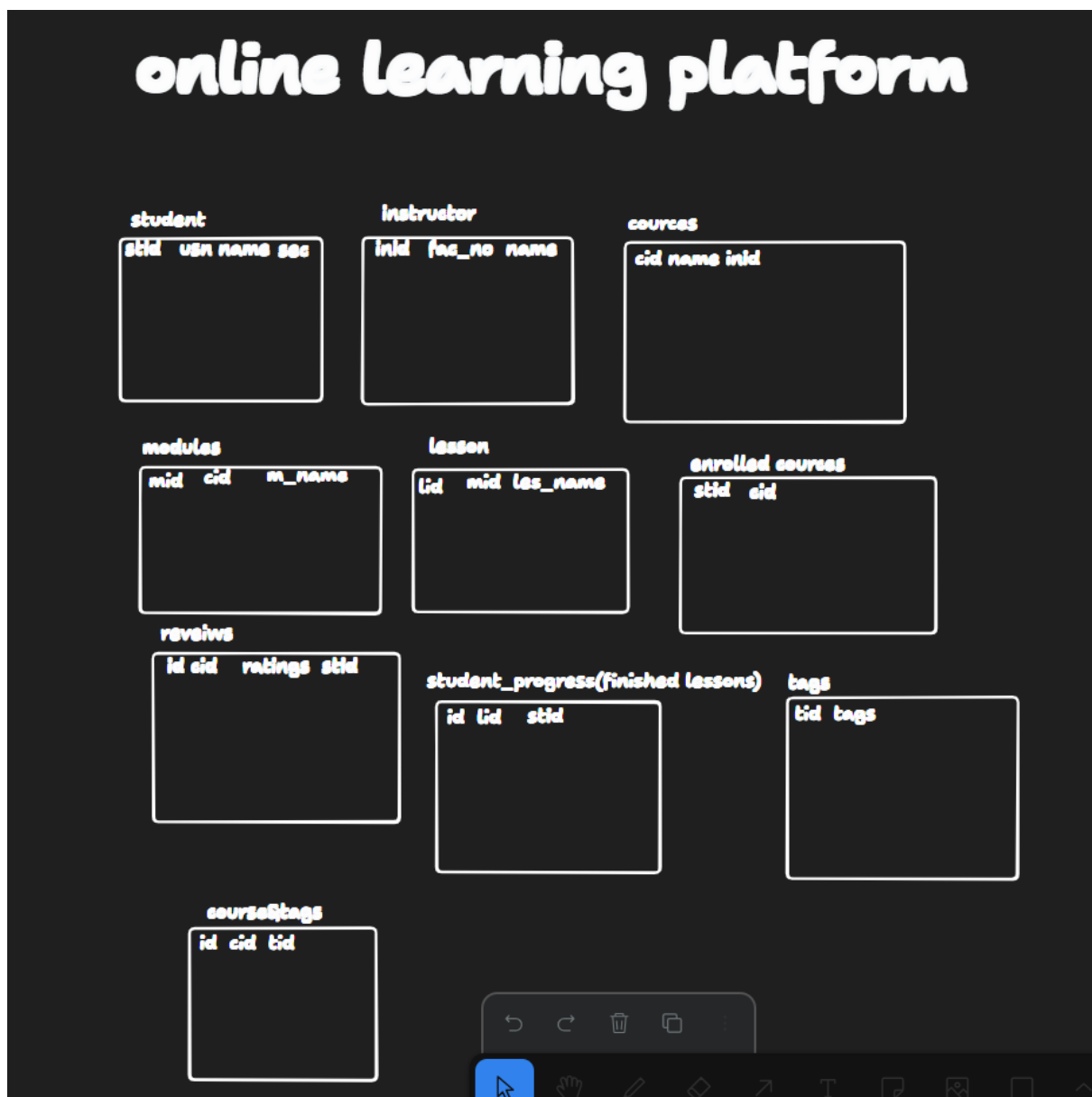
2. Entities and Attributes

- **STUDENT_DET** (stid, usn, st_name, section)
 - **INSTRUCTOR_DET** (instructor_id, facul_no, facul_name)
 - **COURSES** (cid, course_name, instructor_id)
 - **MODULES** (mid, module_name, cid)
 - **LESSON** (lid, lesson_name, mid)
 - **ENROLLMENT** (stid, cid)
 - **REVIEWS** (rid, cid, stid, ratings, review)
 - **PROGRESS** (lid, stid)
 - **TAGS** (tid, tags_name)
 - **COURSE_TAGS** (cid, tid)
-

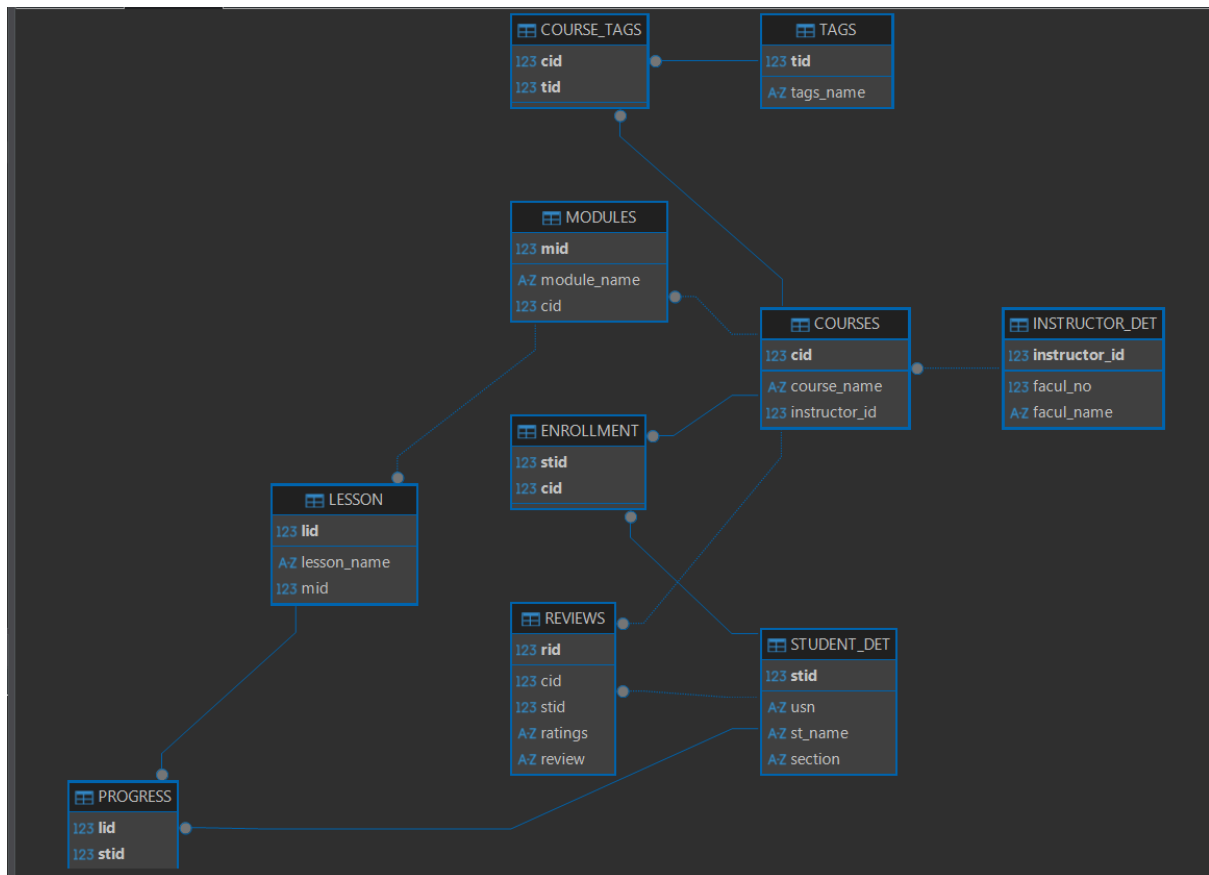
3. Relationships

- One instructor can create multiple courses (1:N)
- One course contains multiple modules (1:N)
- One module contains multiple lessons (1:N)
- Students can enroll in multiple courses and courses can have multiple students (M:N) → handled using **ENROLLMENT**
- Courses can have multiple tags (M:N) → handled using **COURSE_TAGS**
- Students can review courses
- Student progress is tracked lesson-wise

4. ER Diagram



5. Table Schema



6. Normalization

- **1NF**: No repeating groups, all attributes are atomic
- **2NF**: No partial dependency on composite keys
- **3NF**: No transitive dependency (e.g., instructor details separated from courses)

7. Relationship Handling

- **1:N** handled using foreign keys
 - **M:N** handled using junction tables like **ENROLLMENT** and **COURSE_TAGS**
-

Scenario 2: Smart Campus Hostel Management

1. Introduction

This system is designed to manage hostel operations in a university. It handles student accommodation, room allocation, wardens, and maintenance complaints.

The database is designed to efficiently track room assignments and complaint handling while avoiding redundancy.

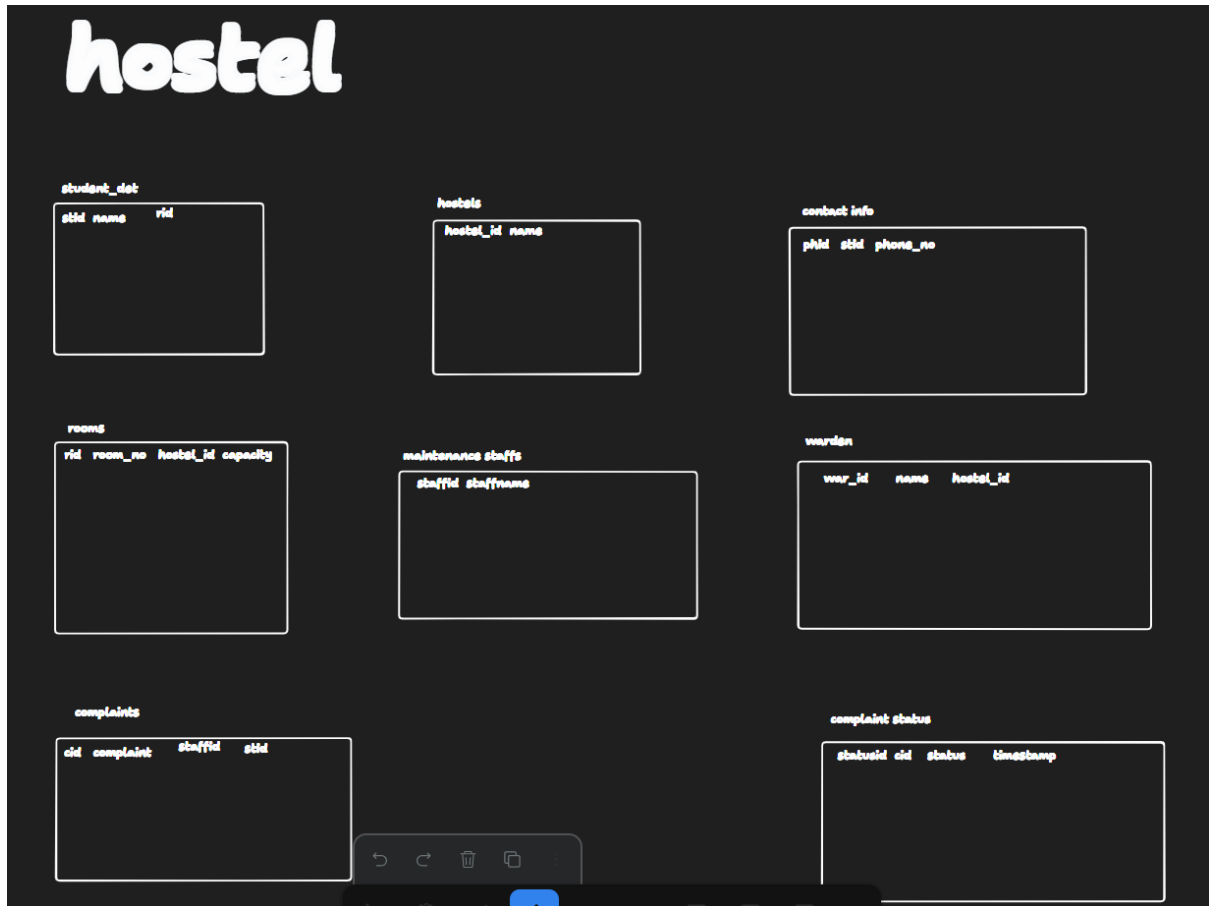
2. Entities and Attributes

- **STUDENTS** (student_id, name, room_id)
 - **HOSTELS** (hostel_id, hostel_name)
 - **ROOMS** (room_id, room_no, hostel_id, capacity)
 - **WARDENS** (warden_id, warden_name, hostel_id)
 - **MAINTENANCE_STAFF** (staff_id, staff_name)
 - **COMPLAINTS** (complaint_id, complaint_text, staff_id, student_id, created_at)
 - **COMPLAINT_STATUS** (status_id, complaint_id, status, updated_at)
 - **STUDENT_PHONE** (phone_id, student_id, phone_no)
-

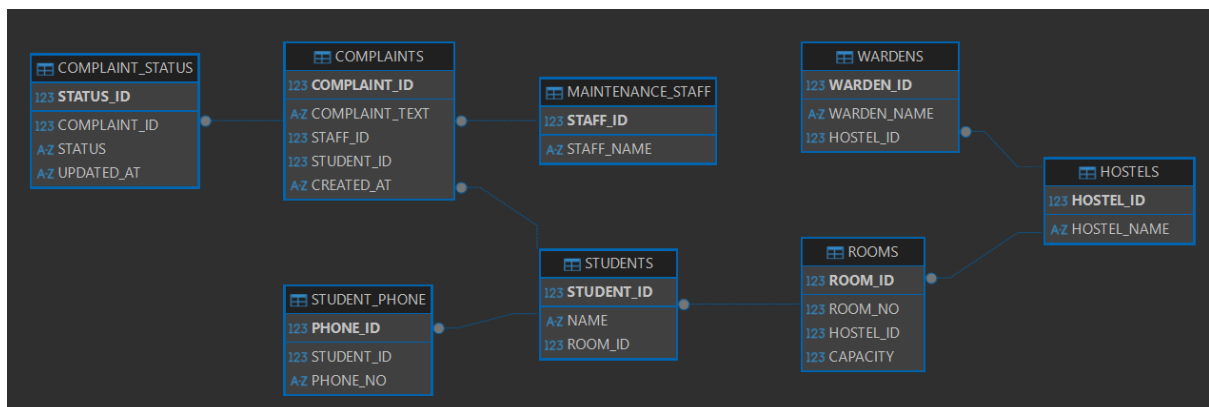
3. Relationships

- One hostel has many rooms (**1:N**)
 - One hostel has many wardens (**1:N**)
 - Students are assigned to rooms
 - One student can raise multiple complaints (**1:N**)
 - Complaints are handled by maintenance staff
 - Complaint status is tracked over time
 - A student can have multiple phone numbers
-

4. ER Diagram



5. Table Schema



6. Normalization

- **1NF**: All attributes are atomic

- **2NF:** No partial dependency
 - **3NF:** No transitive dependency (hostel details not stored in student table)
-

7. Relationship Handling

- **1:N** → Hostel–Room, Student–Complaint
 - **M:N** → Complaint–Staff handled
-